Pandas

Irene Siragusa, PhD

CHILAB
Università degli Studi di Palermo

Pandas

- **Pandas** is a comprehensive library for processing and analyzing multidimensional data. It is based on two main data structures which are the **Series** and the **DataFrame**.
- A Series represents a 1D array of data homogeneous in type, while a DataFrame is a two-dimensional organization of data organized by columns of heterogeneous type.
- Columns can be removed and inserted, and, in general, a dataframe allows for simple groupby, merge, slicing, data indexing, reshaping operations, and allows for handling loading and saving in csv, json, SQL, HTML, Excel and others.
- Mathematical and statistical operations are available on dataframes and apply to all cells or to a user-specified subset.

Pandas

- Pandas is the reference framework for data manipulation in Python

```
$ pip install pandas

$ conda install pandas

import pandas as pd
```

Pandas – Series

```
import pandas as pd

s = pd.Series([1.2, 3.4, float('nan'), 2.1],
index=['a', 'b', 'c', 'd'])

s.dtype          # float64
s.shape          # (4,)
s.index          # Index(['a','b','c','d'])
s.values         # array([1.2, 3.4, nan, 2.1])
s.isna()         # boolean mask NaN
s.describe()     # count, mean, std, min, quartile, max
```

Pandas – DataFrame creation

```
import pandas as pd
import numpy as np

# From list of lists (rows)
df = pd.DataFrame([[1,'Alice',30],[2,'Bob',25]],
columns=['id','name','age'])

# From list of tuples
data = [(1,'Alice',30),(2,'Bob',25)]
df = pd.DataFrame(data, columns=['id','name','age'])
```

Pandas – DataFrame creation

```
# From dictionary: keys = cols names, values = lists
df = pd.DataFrame({
   'id': [1, 2, 3],
   'name': ['Alice', 'Bob', 'Carlo'],
   'age': [30, 25, 35]
})

# From list of dictionaries (rows)
df =
pd.DataFrame([{'id':1,'name':'Alice'},{'id':2,'name':'Bo
b'}])
```

Pandas – DataFrame creation

```
# from Array 2D → DataFrame
arr = np.array([[1,30],[2,25],[3,35]])
df = pd.DataFrame(arr, columns=['id','age'])

# with custom index
df = pd.DataFrame(arr,
columns=['id','age'],
index=['a','b','c'])

# from Series
s = pd.Series([10, 20, 30], name='values')
df = s.to_frame() # Series → DataFrame
```

Pandas – DataFrame attributes

```
df.shape       # (n_rows, n_cols)
df.columns     # Index of colums' names
df.index       # RangeIndex or custom Index personalizzato
df.dtypes      # dtype for each cols
df.values      # array NumPy value
df.size        # total elements (rows*cols)

# Converts types
df['age'] = df['age'].astype(float)
df['date'] = pd.to_datetime(df['date_str'])
```

Pandas – DataFrame import/export

```
# CSV read

df = pd.read_csv('dati.csv')

df = pd.read_csv('dati.csv',
                 sep=';',                # separator
                 header=0,               # header line
                 index_col='id',         # col as index
                 usecols=['a','b'],      # only some cols
                 na_values=['N/A','-'],
                 dtype={'age': int})
```

Pandas – DataFrame import/export

```
# CSV write

df.to_csv('output.csv', index=False)

df.to_csv('output.csv',
          sep=';',
          decimal=',',
          float_format='%.3f',
          encoding='utf-8')
```

Pandas – DataFrame import/export

```
# JSON
df = pd.read_json('data.json')
df.to_json('output.json', orient='records', indent=2)

# Excel
df = pd.read_excel('data.xlsx', sheet_name='Sheet1')
df.to_excel('output.xlsx', sheet_name='Data',
index=False)
```

Pandas – DataFrame selection

```
# Single col → Series
s = df['name']
s = df.name # object notation (no space/keyword)

# More cols → DataFrame
sub = df[['name', 'age']]

# Rows slice
df[2:5]  # rows 2,3,4
df[::-1] # reverse order

# Boolean col
mask = df['age'] > 30 # Boolean series
df[mask] # filtering
```

Pandas – DataFrame selection

```
# loc – label selection
# df.loc[row_label, col_label]
df.loc[0] # row w/ label 0
df.loc[0, 'name'] # single cell
df.loc[[0,1,2], 'name':'age'] # col slice
df.loc[df['age']>30, ['name','age']]

# Assign with loc
df.loc[df['age']>60, 'range'] = 'senior'
```

Pandas – DataFrame selection

```
# iloc – position selection
# df.iloc[row_pos, col_pos]
df.iloc[0]              # 1st row
df.iloc[-1]             # last row
df.iloc[0, 2]           # row 0, col 2

# Slice of rows and cols
df.iloc[0:3]            # firsts 3 rows
df.iloc[:, 1:3]         # 1st and 2nd cols
df.iloc[0:4, 0:2]       # sottoinsieme rettangolare

# List of positions
df.iloc[[0, 2, 4]]      # rows 0, 2, 4
df.iloc[[0,1], [0,2]] # rows 0-1, col 0 and 2
```

Pandas – DataFrame selection

```
# at and iat — (rapid) single cell access

# at → for label (as loc)
val = df.at[3, 'name']     # read
df.at[3, 'name'] = 'Dora' # write

# iat → for position (as iloc)
val = df.iat[3, 1]         # read
df.iat[3, 1] = 'Dora'      # write
```

Pandas – DataFrame filtering

```
# comparison operators: ==, !=, <, >, <=, >=
df[df['age'] > 30]
df[df['name'] == 'Alice']

# Combination with & (and), | (or), ~ (not)
df[(df['age'] > 25) & (df['age'] < 40)]
df[(df['name']=='Alice') | (df['name']=='Bob')]
df[~(df['age'] > 60)] # negation
```

Pandas – DataFrame filtering

```
# isin e between
df[df['name'].isin(['Alice','Bob','Carlo'])]
df[df['age'].between(20, 35)]
# str accessor (textual cols)
df[df['name'].str.startswith('A')]
df[df['name'].str.contains('li', case=False)]
# query() — SQL-like syntax
df.query('age > 25 and name != "Bob"')
```

Pandas – analysis

```
df.head(5)  # first 5 rows (default)
df.tail(3)  # last 3 rows

df.info()   # dtype, non-null count, memory usage

# Count values per col
df['genre'].value_counts()
df.nunique()          # unique value per col
df['col'].unique() # array with unique values
```

Pandas – analysis

```
# describe() – numeric statistics of cols numeriche
df.describe() # count, mean, std, min, quartile, max
df.describe(include='all') # include categorical
features
df.describe(percentiles=[.1,.25,.75,.9])

# Single statistics
df['age'].mean()
df['age'].median()
df['age'].std() # standard deviation
```

Pandas – analysis

```
# Overall statistics
df.mean()
df.median()
df.std() # standard deviation

# Correlation .corr()
# Pearson (default)
corr = df.corr(numeric_only=True)
corr['age'] # correlations with age
df.corr(method='spearman')
```

Pandas – missing values

```
# Missing values (NaN or None)
df.isna()                    # boolean DataFrame
df.isnull()                  # alias of isna()
df.isna().sum()              # NA per col
df.isna().sum(axis=1)        # NA per row
df.isna().any()              # True if at least 1 NA
df.isna().mean() * 100       # percentage NA
```

Pandas – duplicates

```
# duplicated rows → Boolean Series
df.duplicated() # True per each duplicate
df.duplicated(keep='first') # keep 1st occurence
df.duplicated(keep=False) # sing ALL the duplicates

df.duplicated(subset=['name','age']) # of cols subset

# count and show duplicates
df.duplicated().sum() # how many duplicates
df[df.duplicated(keep=False)]# visualize duplicates
```

Pandas – cleaning

```
# Remove rows with at least one NA
df_clean = df.dropna()
df_clean = df.dropna(axis=0) # across rows (default)
df_clean = df.dropna(axis=1) # rimuove cols con NA
df.dropna(thresh=3) # keep rows with ≥3 no-NA values
df.dropna(subset=['name','age']) # only these cols
```

Pandas – cleaning

```
# Fill NA values

# costant value
df.fillna(0) # fill all with 0
df['age'].fillna(df['age'].mean()) # with mean

# Propagation
df.fillna(method='ffill') # forward fill
df.fillna(method='bfill') # backward fill
df.fillna(method='ffill', limit=2) # max 2 propagations
df.interpolate() # linear interpolation
```

Pandas – cleaning

```
# Add col
df['bmi'] = df['weight'] / df['height']**2
df['range'] = 'standard' # constant value

# insert() – specific position
df.insert(2, 'bmi', df['weight']/df['height']**2)

# Remove cols
df = df.drop(columns=['id','extra'])
df = df.drop(columns='id') # single col
```

Pandas – cleaning

```
# Add row – pd.concat
nuova = pd.DataFrame([{'name':'Elena','age':28}])
df = pd.concat([df, nuova], ignore_index=True)

# Remove rows per label
df = df.drop(index=3)            # single row
df = df.drop(index=[1, 3, 5])    # more rows

# Remore per condition
df = df[df['age'] >= 18]         # filtering
df = df.drop(df[df['age']<18].index)
```

Pandas – where()

```
# where() keep where True, NaN otherwise
df['age'].where(df['age'] >= 18) # <18 → NaN
df['age'].where(df['age'] >= 18, other=0) # <18 → 0

# mask() substitue where True
df['age'].mask(df['age'] > 100, other=np.nan)

# Apply to the entire DataFrame
df.where(df > 0)            # negatives → NaN
df.mask(df < 0, other=0)    # negatives → 0
```

Pandas – any() and all()

```
# any() True it at least one element is True
df.isna().any()          # cols with at least 1 NA
df.isna().any(axis=1)  # rows with at least 1 NA

# all() True if all elements are True
df['age'].gt(0).all()      # all ages > 0
(df > 0).all(axis=1)       # all cols > 0 per row

# Combination with filtering
df[df.gt(0).all(axis=1)] # rows with all values > 0
df.dropna(how='any’)       # remove rows with any NA
```

Pandas – groupby()

```
# groupby() aggregation
g = df.groupby('department')

# Aggregated functions
g['salary'].mean() # media per department
g['salary'].sum()
g.size()           # size of each group
g['age'].agg(['mean','std','min','max'])

# group for multiple cols
df.groupby(['department','genre'])['salary'].mean()
```

Pandas – apply()

```
# apply a custom function over Series (per element)
df['name'] = df['name'].apply(str.upper)
df['age'] = df['age'].apply(lambda x: x*2)

# apply over DataFrame (for col or row)
df.apply(np.sum)          # sum per col (axis=0)
df.apply(np.sum, axis=1)  # sum per row

# apply with GroupBy
def zscore(x):
    return (x - x.mean()) / x.std()
df.groupby('department')['salary'].apply(zscore)
```

Pandas – concat()

```
# vertical concatenation
df_tot = pd.concat([df1, df2, df3], ignore_index=True)

# horizontal concatenation
df_tot = pd.concat([df1, df2], axis=1)
```

Pandas – plot

```
import matplotlib.pyplot as plt
df['age'].plot(kind='hist', bins=20)
df.plot(x='age', y='salary', kind='scatter')
df['department'].value_counts().plot(kind='bar')
df[['salary','age']].plot(kind='box')
df['salary'].plot(kind='kde') # density

# Scatter plot
from pandas.plotting import scatter_matrix
scatter_matrix(df, diagonal='hist', figsize=(8,8))
```

Pandas – principal module and class

Module / Class	Description
`pd.DataFrame`	2-d tabular data-structure
`pd.Series`	1-d labeled array
`pd.read_csv / read_excel`	read from file CSV, Excel, JSON, SQL…
`pd.DataFrame.fillna`	manage missing values
`pd.DataFrame.dropna`	removes rows/colomns with NaN values
`pd.get_dummies`	one-hot encoding of categorical values

Pandas – principal module and class

Module / Class	Description
`pd.cut / pd.qcut`	binning of continuous variables
`pd.DataFrame.groupby`	aggregation over groups
`pd.DataFrame.merge / join`	union of DataFrames
`pd.DataFrame.apply`	allow application of custom functions
`pd.DataFrame.describe`	descriptive statistics of the DataFrame
`pd.DataFrame.info`	descriptive of data type and NaN count in the DataFrame

Pandas

- Documentation: https://pandas.pydata.org/docs/
- User Guide: https://pandas.pydata.org/docs/user_guide/
- API Reference: https://pandas.pydata.org/docs/reference/
- 10 minutes to pandas: https://pandas.pydata.org/docs/user_guide/10min.html
- Cheat Sheet: https://pandas.pydata.org/Pandas_Cheat_Sheet.pdf
- Cookbook: https://pandas.pydata.org/docs/user_guide/cookbook.html
- Visualization integrata: https://pandas.pydata.org/docs/user_guide/visualization.html

Hands on!

- Analyze the data set
- Infer missing data
- https://www.kaggle.com/datasets/himelsarder/retail-product-dataset-with-missing-values